

---

## HDOC – Day 4

### C Programming Assignment

*Multiply Two Numbers Without Using the \* Operator*

|               |                    |
|---------------|--------------------|
| <b>Name</b>   | Rudra Pratap Singh |
| <b>SAP ID</b> | 590032811          |
| <b>Batch</b>  | 16                 |
| <b>Date</b>   | 29 August 2026     |

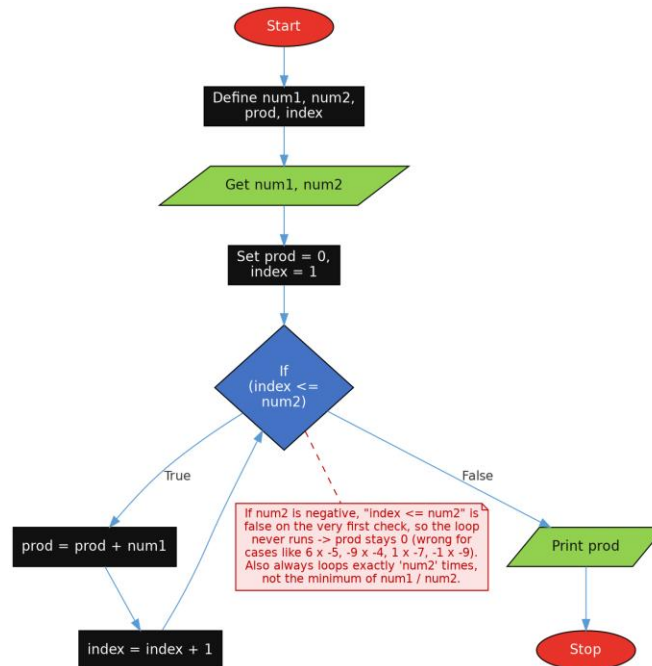
## 1. Problem Statement

Find the product of two integers without using the multiplication operator ( \* ), using the logic of repeated addition. The solution must correctly handle all sign combinations (positive  $\times$  positive, positive  $\times$  negative, negative  $\times$  positive, negative  $\times$  negative) and multiplication by zero, while using the minimum possible number of additions.

## 2. Given Algorithm (Incorrect)

```
Define num1, num2, prod, index as integers
Get num1, num2
Set prod = 0, index = 1
Repeat steps 4.1 and 4.2 while (index <= num2):
    4.1 prod = prod + num1
    4.2 index = index + 1
Print prod
Stop
```

## 3. Given Flowchart (Incorrect)



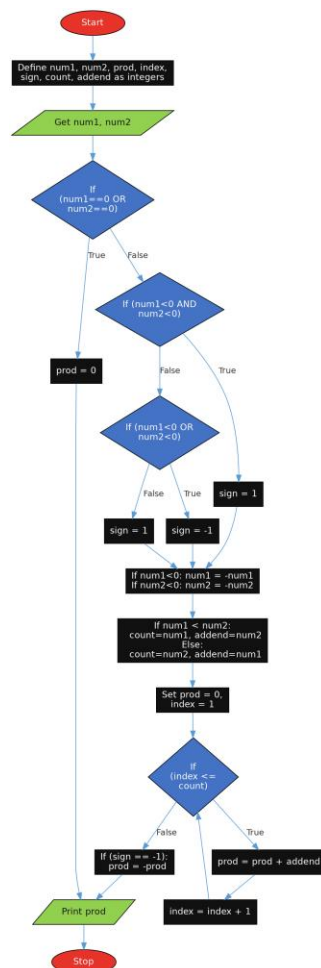
## 4. Mistakes Identified

- Loop condition breaks for negative num2: index starts at 1 and only increases, so "index <= num2" is FALSE on the very first check whenever num2 is negative. The loop body never executes, and prod is wrongly left at 0 (fails for test cases like  $6 \times -5$ ,  $-9 \times -4$ ,  $1 \times -7$ ,  $-1 \times -9$ ).
- No explicit sign handling: the algorithm relies purely on num1's own sign propagating through repeated addition. This happens to work when num2 is positive, but breaks completely as soon as num2 is negative (see bullet above).
- Not the minimum number of additions: the loop always runs exactly num2 times, even when num1 has the smaller magnitude. For example,  $3 \times 12$  should need only 3 additions, but this algorithm performs 12.

## 5. Corrected Algorithm

1. Define num1, num2, prod, index, sign, count, addend as integers.
2. Get num1, num2.
3. If num1 = 0 OR num2 = 0, set prod = 0 and go to Step 12.
4. If num1 < 0 AND num2 < 0, set sign = 1 (negative × negative gives a positive answer).
5. Otherwise, if num1 < 0 OR num2 < 0, set sign = -1 (exactly one operand is negative).
6. Otherwise (both positive), set sign = 1.
7. If num1 < 0, set num1 = -num1. If num2 < 0, set num2 = -num2 (this makes both numbers positive).
8. If num1 < num2, set count = num1 and addend = num2; otherwise set count = num2 and addend = num1 (using the smaller value as the loop counter guarantees the minimum number of additions).
9. Set prod = 0, index = 1.
10. Repeat while (index ≤ count): prod = prod + addend; index = index + 1.
11. If sign = -1, set prod = -prod.
12. Print prod.
13. Stop.

## 6. Corrected Flowchart



## 7. C Code

### Incorrect version (direct translation of the given algorithm):

```
#include <stdio.h>

/* INCORRECT VERSION
- The loop condition "index <= num2" is checked with index starting at 1.
  If num2 is negative, this condition is false on the very first check,
  so the loop body never executes and the function wrongly returns 0.
- The loop always runs exactly num2 times (never the smaller of the two
  operands), so it does not use the minimum possible number of additions.
*/
int multiply(int num1, int num2) {
    int prod = 0;
    int index = 1;
    while (index <= num2) {
        prod = prod + num1;
        index = index + 1;
    }
    return prod;
}

int main(void) {
    int num1, num2;
    printf("Enter num1 and num2: ");
    scanf("%d %d", &num1, &num2);
    printf("Product: %d\n", multiply(num1, num2));
    return 0;
}
```

### Corrected version:

```
#include <stdio.h>

/* CORRECTED VERSION
- Explicitly handles num1 = 0 or num2 = 0
- Works out the sign of the answer with simple if-else statements
- Makes both numbers positive before looping
- Loops over the SMALLER number (count) and adds the LARGER number
  (addend) each time -> minimum possible number of additions
*/
int multiply(int num1, int num2) {
    int prod, index, sign, count, addend;

    if (num1 == 0 || num2 == 0)
        return 0;

    /* Step 1: work out the sign of the answer */
    if (num1 < 0 && num2 < 0) {
        sign = 1;          /* negative x negative = positive */
    } else if (num1 < 0 || num2 < 0) {
        sign = -1;         /* exactly one of them is negative */
    } else {
        sign = 1;          /* both positive */
    }

    /* Step 2: make both numbers positive so the loop below is simple */
    if (num1 < 0)
        num1 = -num1;
    if (num2 < 0)
        num2 = -num2;
```

```

/* Step 3: loop the smaller number of times -> fewer additions */
if (num1 < num2) {
    count = num1;
    addend = num2;
} else {
    count = num2;
    addend = num1;
}

prod = 0;
index = 1;
while (index <= count) {
    prod = prod + addend;
    index = index + 1;
}

if (sign == -1)
    prod = -prod;

return prod;
}

int main(void) {
    int num1, num2;
    printf("Enter num1 and num2: ");
    scanf("%d %d", &num1, &num2);
    printf("Product: %d\n", multiply(num1, num2));
    return 0;
}

```

## 8. Verification Against Given Test Cases

The corrected C program was compiled and executed against all 13 test cases supplied in the assignment. Every actual output matched the expected output and used exactly the minimum number of additions specified.

| TC# | Input (num1, num2) | Expected Product | Actual Product | Min. Additions (Expected) | Actual Additions | Result |
|-----|--------------------|------------------|----------------|---------------------------|------------------|--------|
| 1   | 12, 3              | 36               | 36             | 3                         | 3                | Pass   |
| 2   | 3, 12              | 36               | 36             | 3                         | 3                | Pass   |
| 3   | -6, 5              | -30              | -30            | 5                         | 5                | Pass   |
| 4   | 6, -5              | -30              | -30            | 5                         | 5                | Pass   |
| 5   | -9, -4             | 36               | 36             | 4                         | 4                | Pass   |
| 6   | -4, -9             | 36               | 36             | 4                         | 4                | Pass   |
| 7   | 0, 8               | 0                | 0              | 0                         | 0                | Pass   |
| 8   | 8, 0               | 0                | 0              | 0                         | 0                | Pass   |
| 9   | 0, -7              | 0                | 0              | 0                         | 0                | Pass   |
| 10  | -7, 0              | 0                | 0              | 0                         | 0                | Pass   |
| 11  | 0, 0               | 0                | 0              | 0                         | 0                | Pass   |
| 12  | 1, -7              | -7               | -7             | 1                         | 1                | Pass   |
| 13  | -1, -9             | 9                | 9              | 1                         | 1                | Pass   |

## 9. Conclusion

The given algorithm's core flaw is that its loop bound, "index <= num2", is only ever satisfied when num2 is positive; a negative num2 makes the loop skip entirely and silently returns 0. It also does not minimize additions, since it always iterates num2 times regardless of which operand is smaller. The corrected version fixes both issues: it works out the sign of the answer with simple if-else checks, converts both numbers to positive, and always loops the smaller of the two magnitudes while adding the larger one.